

Tanzeem Kutub Khana



FIREBASE + ADMOB Complete Beginner Setup Guide

Explained step-by-step like to a child — every click, every paste.

Updated for 2026 Firebase Console

For: Mr. M. Asad | A.A.A. COMMUNITY

Jamia Rasoolia Sherazia, Lahore

This guide is in English. A Roman-Urdu translation
(Urdu Tarjuma) is provided at the very end.

What This Guide Will Do For You

Right now your app **Tanzeem Kutub Khana** works fully offline — every book, paper, cover image and PDF is stored inside the phone. That is good, but it means: whatever YOU (the admin) add on your phone, other users do NOT see it.

Firebase is a FREE online database from Google. When we connect it, the flow becomes:

The Goal (very important)

- Admin opens the app → adds/edits a Book, Paper, Notice or Ad.
- That change is saved to Firebase (the online database in the cloud).
- EVERY user's app reads from Firebase → they instantly see your change.
- So the moment YOU upload something, ALL users get it. This is called 'responsive / real-time' data.

Follow this guide slowly. Do **one step at a time**. You cannot break anything — if you make a mistake you can always delete and redo. Let's begin.

★ *TIP: You only need a Gmail account and a web browser (Chrome). Everything here is 100% free.*

Chapter 1 — Create Your Firebase Account & Project

Step 1: Open the Firebase website

1. Open Google Chrome on your computer.
2. In the address bar type this and press Enter:

```
https://console.firebase.google.com
```

3. If it asks you to Sign in, click Sign in and log in with your Gmail (email + password).

Step 2: Create a new project

1. You will see a big card that says “Create a project” (or “Add project”). Click it.
2. A box appears: “Enter your project name”. Type exactly:

```
Tanzeem Kutub Khana
```

3. Below the name Firebase shows a small ID like tanzeem-kutub-khana-xxxx. Leave it as it is. Click Continue.
4. Next screen: “Google Analytics”. You can turn the switch OFF (we do not need it). Click Continue / Create project.
5. Wait 30–60 seconds. When you see “Your new project is ready”, click Continue.

★ *TIP: Congratulations — your project is made! Now you are inside the Firebase Console dashboard.*

Chapter 2 — Register the Android App

Now we tell Firebase about your Android app so it can give us a config file.

Step 1: Add the Android app

1. On the project home page, near the top you see small icons: a web icon `</>`, an Apple icon, and an Android robot icon. Click the Android robot icon.
2. A form opens: “Register app”.
3. In the box “Android package name” type EXACTLY (this must match the app, do not change even one letter):

```
com.tanzeemkutub.books
```

4. App nickname (optional): type Tanzeem Kutub.
5. Leave the SHA-1 box empty. Click Register app.

Step 2: Download google-services.json

1. The next screen says “Download google-services.json”. Click the blue Download google-services.json button.
2. The file google-services.json saves to your Downloads folder. KEEP THIS FILE SAFE.
3. Click Next, then Next again, then Continue to console. (Ignore all the code they show — we already did that part in the app.)

What to do with google-services.json

- Send this file to your developer, OR
- Upload it in the Firebase tab of your app-builder, OR
- Place it inside the app folder at: android/app/google-services.json
- This file is the 'key' that connects the app to YOUR project.

★ *TIP: Every time you make a NEW Firebase project you get a NEW google-services.json. Always use the latest one.*

Chapter 3 — Create the Firestore Database

This is the actual online database where your books and papers will live.

Step 1: Open Firestore

1. On the LEFT side menu, find and click Build (it opens a small list).
2. In that list click Firestore Database.
3. In the middle of the page click the blue button Create database.

Step 2: Choose Edition & Mode

1. If it asks for an Edition, choose Standard edition (the free one). Click Continue.
2. It may ask for mode: choose Native mode (if shown). Click Continue.

Step 3: Choose Location

1. Now it asks “Location”. Click the dropdown and choose one close to Pakistan:

asia-south1 (Mumbai)

2. Click Continue / Next.

Step 4: Choose Rules mode & finish

1. It asks: Production mode or Test mode. Choose Start in production mode.
2. Click Create (or Enable).
3. Wait about a minute. When the empty database table appears — DONE! Your database is live.

★ *TIP: Do not worry that it says 'production mode' — in Chapter 4 we will paste the correct Rules so your app can read/write.*

Chapter 4 — Security Rules (copy & paste)

Rules decide WHO can read and write. We will use simple, safe rules: **anyone can READ** (so all users see books), but **only logged-in admins can WRITE** (add/edit/delete).

Step 1: Open the Rules tab

1. You are inside Firestore Database. At the TOP you see tabs: Data, Rules, Indexes, Usage.
2. Click the Rules tab.
3. You will see a code box with some text. Select ALL the text inside (press Ctrl+A while your mouse is in the box) and DELETE it.

Step 2: Paste these EXACT rules

Now paste this whole block into the empty box:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper: is this a signed-in admin?
    function isAdmin() {
      return request.auth != null
        && exists(/databases/{database}/documents/admins/{request.auth.uid});
    }

    // Books, Papers, Notices, Ads: everyone can READ,
    // only admins can WRITE.
    match /books/{doc}    { allow read: if true; allow write: if isAdmin(); }
    match /papers/{doc}   { allow read: if true; allow write: if isAdmin(); }
    match /notices/{doc}  { allow read: if true; allow write: if isAdmin(); }
    match /ads/{doc}      { allow read: if true; allow write: if isAdmin(); }

    // Users: a user can read/write only their OWN profile.
    match /users/{uid} {
      allow read: if true;
      allow write: if request.auth != null && request.auth.uid == uid;
    }

    // Admins list: readable, but only editable from the Firebase Console.
    match /admins/{uid} {
      allow read: if true;
      allow write: if false;
    }
  }
}
```

Step 3: Publish

1. Above the box click the blue Publish button.
2. If it asks to confirm, click Publish again. Rules are now LIVE.

Step 4: Make YOURSELF the admin

Because only admins can write, you must add your own account to an **admins** list. First you need your User ID (UID) — you will get it in Chapter 7 after enabling login. For now just remember this

step:

1. Later, go to Firestore Database → Data tab.
2. Click + Start collection, name it exactly: admins.
3. Add a document whose Document ID is YOUR UID (from Chapter 7).
4. Inside it add one field: name = role, type = string, value = admin. Save.

★ *TIP: If you want a QUICK TEST first (everyone can write), you can temporarily use: allow read, write: if true; — but change it back to the safe rules above before real users use the app.*

Chapter 5 — The Collections Your App Needs

A 'collection' is like a folder / table. Inside it are 'documents' (single records). Below are ALL the collections this app uses, with EVERY field, its type, and what it means. Create them exactly like this.

1) books (PDF books, sharh, tarjuma)

Fields for each book document

- id — string (unique id, e.g. book_001)
- name — string (book title, e.g. Sahih Bukhari)
- author — string (writer name)
- classId — string (which shuba/class it belongs to)
- subject — string (subject name)
- coverPath — string (cover image LINK, https://...jpg)
- pdfPath — string (the book PDF LINK, https://...pdf)
- isPaper — boolean (false for normal books)
- year — string (optional, e.g. 2026)
- sharh — array (list of sharh: each has title, authorName, pdfPath)
- translations — array (list of tarjuma: title, authorName, pdfPath)
- totalPages — number (optional)

Sharah & Tarjuma are stored INSIDE the book as arrays. Each item is a small object with: **id** (string), **title** (string), **authorName** (string), **pdfPath** (string link). So one book can have its Arabic PDF + many sharh PDFs + many tarjuma PDFs, all with cover + links.

2) papers (exam / guess papers)

Fields for each paper document

- id — string
- name — string (paper title)
- classId — string (which shuba)
- subject — string
- coverPath — string (cover image LINK)
- pdfPath — string (paper PDF LINK)
- isPaper — boolean (true)
- paperType — string (e.g. Annual, Model, Guess)
- year — string (e.g. 2026)

3) notices (Notifications from Admin)

Fields for each notice document

- id — string
- title — string (short heading)
- body — string (the message text, Urdu allowed)
- createdAt — number (time in milliseconds)
- active — boolean (true = show to users)

When admin adds a notice here, all users' apps read it and show the notification.

4) ads (Private Ads from Admin)

Fields for each ad document

- id — string
- title — string
- imagePath — string (ad banner image LINK)
- linkUrl — string (where it opens when tapped)
- type — string (image or video)
- active — boolean (true = show)
- createdAt — number

5) users (User Info)

Fields for each user document (Document ID = user's UID)

- uid — string (auto from login)
- name — string
- email — string
- classId — string (their selected shuba)
- city — string (optional)
- createdAt — number

6) admins (who is allowed to edit)

Fields for each admin document (Document ID = admin's UID)

- role — string (value: admin)

★ *TIP: You do NOT have to type all six now. Start with 'books' and 'papers'. Add the others when you need them. Collections are created automatically the first time you add a document to them.*

Chapter 6 — Add Your First Book (every click)

Let's add one real book so you learn how. After this, adding books is the same every time.

1. Go to Firestore Database → Data tab.
2. Click + Start collection.
3. In Collection ID type exactly: **books** → click Next.
4. Now it asks for the first document. In Document ID click Auto-ID (Firebase makes a random id) OR type your own like **book_001**.
5. Now add fields ONE BY ONE. For each field: type the Field name, pick the Type, type the Value, then click + Add field for the next one:
 - Field: **name** Type: **string** Value: **Sahih Bukhari**
 - Field: **author** Type: **string** Value: **Imam Bukhari**
 - Field: **classId** Type: **string** Value: **daura_hadith**
 - Field: **subject** Type: **string** Value: **Hadith**
 - Field: **coverPath** Type: **string** Value: **https://example.com/cover.jpg**
 - Field: **pdfPath** Type: **string** Value: **https://example.com/book.pdf**
 - Field: **isPaper** Type: **boolean** Value: **false**
6. Click Save. Your first book is now online! Every user's app will see it.

How to add a Sharh or Tarjuma to that book

1. Open the book document you just made.
2. Click + Add field. Field name: **sharah**, Type: **array**.
3. Inside the array click + Add item, choose type **map**, and inside the map add: **id** (string), **title** (string), **authorName** (string), **pdfPath** (string link). Save.
4. For tarjuma do the same but name the field **translations**.

★ TIP: *coverPath* and *pdfPath* must be public LINKS ending in .jpg / .png / .pdf. You can host PDFs on Google Drive (set 'Anyone with link'), Firebase Storage, or any website.

Chapter 7 — Turn On Login (Authentication)

Login lets users have accounts and lets US know who the admin is.

1. **LEFT menu → Build → Authentication.**
2. **Click Get started.**
3. **You see a list of sign-in methods. Click Email/Password.**
4. **Turn the first switch ON (Enable). Click Save.**

How to find YOUR UID (for the admins collection)

1. **First create an account by signing up inside the app with your email + password.**
2. **Come back to Authentication → Users tab. You will see your email in the list.**
3. **Copy the long User UID shown next to your email.**
4. **Go to Firestore → Data → create collection admins → add a document whose ID is that UID → field role = admin (as in Chapter 4). Now you are the admin!**

★ *TIP: Never share your admin email/password. Only admins can add or change books.*

Chapter 8 — Is It Really Free? (Spark Plan)

Yes. Firebase's free **Spark plan** is enough for a library app. Daily free limits:

Free daily limits (Spark plan)

- 50,000 reads per day (opening/viewing data)
- 20,000 writes per day (adding/editing data)
- 20,000 deletes per day
- 1 GB stored data total
- 10 GB/month network transfer

For a madrasa library this is plenty. You do NOT need to add a credit card for the Spark plan. If one day you grow very large, you can upgrade — but you will likely never need to.

★ *TIP: To reduce reads: the app already caches data. Users only re-download when something changes.*

Chapter 9 — AdMob: Create Account & Ad Units

AdMob shows Google ads in your app and pays you money. This is separate from Firebase.

Step 1: Create the AdMob account

1. Open a new tab and go to:

```
https://admob.google.com
```

2. Click Get started / Sign in with the SAME Gmail. Fill your country = Pakistan, accept terms.

Step 2: Add your app to AdMob

1. LEFT menu → Apps → Add app.
2. It asks 'Is your app listed on Google Play?' — if not published yet choose No.
3. Platform: choose Android. App name: Tanzeem Kutub Khana. Click Add app.
4. You now get an App ID that looks like this — COPY it:

```
ca-app-pub-XXXXXXXXXXXXXXXX~YYYYYYYYYY
```

Step 3: Create a Banner Ad Unit

1. Click your app → Ad units → Add ad unit.
2. Choose Banner.
3. Name it Home Banner. Click Create ad unit.
4. You get an Ad unit ID like this — COPY it:

```
ca-app-pub-XXXXXXXXXXXXXXXX/ZZZZZZZZZZ
```

★ TIP: The App ID has a ~ (tilde). The Ad unit ID has a / (slash). Do not mix them up!

Chapter 10 — AdMob in the App (responsive)

Give these two IDs to your developer, OR if editing the code yourself, here is EXACTLY what to add. The banner below is **adaptive** — it automatically fits every screen size (responsive).

A) pubspec.yaml — add the package

```
dependencies:  
  google_mobile_ads: 5.3.1
```

B) AndroidManifest.xml — add your App ID

Inside <application> ... </application> add:

```
<meta-data  
  android:name="com.google.android.gms.ads.APPLICATION_ID"  
  android:value="ca-app-pub-XXXXXXXXXXXXXXXX~YYYYYYYYYY"/>
```

C) main.dart — initialize once

```
import 'package:google_mobile_ads/google_mobile_ads.dart';  
  
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await MobileAds.instance.initialize();  
  runApp(const MyApp());  
}
```

D) A responsive (adaptive) banner widget

```

class AdaptiveBanner extends StatefulWidget {
  const AdaptiveBanner({super.key});
  @override
  State<AdaptiveBanner> createState() => _AdaptiveBannerState();
}

class _AdaptiveBannerState extends State<AdaptiveBanner> {
  BannerAd? _ad;
  bool _loaded = false;

  // TEST id (safe). Replace with your real Ad unit ID for release.
  static const _unitId = 'ca-app-pub-3940256099942544/6300978111';

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    _load();
  }

  Future<void> _load() async {
    final width = MediaQuery.of(context).size.width.truncate();
    final size = await AdSize
      .getCurrentOrientationAnchoredAdaptiveBannerAdSize(width);
    if (size == null) return;
    final ad = BannerAd(
      size: size,
      adUnitId: _unitId,
      request: const AdRequest(),
      listener: BannerAdListener(
        onAdLoaded: (_) => setState(() => _loaded = true),
        onAdFailedToLoad: (ad, e) => ad.dispose(),
      ),
    )..load();
    _ad = ad;
  }

  @override
  void dispose() { _ad?.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) {
    if (!_loaded || _ad == null) return const SizedBox.shrink();
    return SizedBox(
      width: _ad!.size.width.toDouble(),
      height: _ad!.size.height.toDouble(),
      child: AdWidget(ad: _ad!),
    );
  }
}

```

Then just place **const AdaptiveBanner()** at the bottom of any screen. It resizes itself for every phone = responsive.

IMPORTANT AdMob safety rules

- While TESTING always use the test IDs above — Banner test: ca-app-pub-3940256099942544/6300978111
- NEVER click your own ads — Google can ban your account.
- Use your REAL Ad unit ID only in the final release build.
- It can take a few hours after account approval before real ads show.

Chapter 11 — Final Checklist

Firestore

- ☐ Created Firestore project
- ☐ Registered Android app (package com.tanzeemkutub.books)
- ☐ Downloaded NEW google-services.json and gave it to the app
- ☐ Created Firestore Database (Standard, asia-south1, production)
- ☐ Pasted the Rules and clicked Publish
- ☐ Enabled Email/Password login
- ☐ Added my UID to the 'admins' collection
- ☐ Added at least 1 book — it appears in the app

AdMob

- ☐ Created AdMob account
- ☐ Added Android app, got App ID (~)
- ☐ Created a Banner ad unit, got Ad unit ID (/)
- ☐ Put App ID in AndroidManifest.xml
- ☐ Added AdaptiveBanner widget
- ☐ Tested with TEST ids, then switched to REAL ids for release

Once all boxes are ticked, whatever the admin uploads (books, papers, sharh, tarjuma, notices, ads) will appear for ALL users instantly — exactly the responsive system you asked for.

URDU TARJUMA

(Roman Urdu mein aasan khulasa)

Yeh Guide Kya Karti Hai: Abhi aap ki app 'Tanzeem Kutub Khana' sirf phone ke andar kaam karti hai. Jo aap admin add karte hain woh doosre users ko nazar nahi aata. **Firestore** Google ka MUFT online database hai. Isko connect karne ke baad: jab bhi ADMIN koi book, paper, notice ya ad upload karega, woh online database (cloud) mein save hoga, aur HAR user ki app woh cheez foran dekh legi. Isko 'responsive / real-time' data kehte hain.

Chapter 1 — Account & Project: Chrome kholein, likhein console.firebase.google.com, Gmail se sign in karein. 'Create a project' par click karein, naam likhein 'Tanzeem Kutub Khana', Continue dabaein, Analytics OFF karein, project ban jaega.

Chapter 2 — Android App: Android (robot) icon par click karein. Package name likhein bilkul yeh: **com.tanzeemkutub.books**. 'Register app' dabaein. Phir **google-services.json** file download karein aur developer ko dein ya app ke Firebase tab mein upload karein. (Naye project ka naya file hamesha use karein.)

Chapter 3 — Database Banana: Left menu → Build → Firestore Database → 'Create database'. Edition: **Standard**. Location: **asia-south1 (Mumbai)**. Mode: **Production**. 'Create' dabaein. Database taiyar!

Chapter 4 — Rules (Qawaid): Firestore ke upar **Rules** tab kholein, andar ka poora text mitaayein, aur guide (English wala Chapter 4) mein diya gaya Rules ka block paste karein, phir **Publish** dabaein. In rules mein: sab log books PARH sakte hain, lekin sirf ADMIN edit kar sakta hai. Khud ko admin banane ke liye ek collection 'admins' banaein aur us mein apni UID daalein (UID Chapter 7 se milegi).

Chapter 5 — Collections (Folders): App ko yeh chahiye: **books** (kitaben + sharah + tarjuma with cover aur PDF links), **papers** (papers with cover aur link), **notices** (admin ki taraf se notification), **ads** (admin ke private ads), **users** (users ki maloomat), aur **admins** (kaun edit kar sakta hai). Sharah aur Tarjuma book ke andar hi 'array' ke tor par save hoti hain (har ek mein title, authorName, aur PDF link).

Chapter 6 — Pehli Book Add Karna: Data tab → '+ Start collection' → naam 'books' → Next. Document ID Auto-ID chunein. Phir ek ek field add karein: name, author, classId, subject, coverPath (image link), pdfPath (PDF link), isPaper=false. 'Save' dabaein. Book online ho gayi — har user dekhega! Sharah/Tarjuma ke liye book ke andar 'sharah' aur 'translations' naam ke array bana kar unme title/author/pdf link daalein.

Chapter 7 — Login On Karna: Build → Authentication → 'Get started' → 'Email/Password' ko ON karke Save. Apni UID lene ke liye pehle app mein apne email/password se account banaein, phir Authentication → Users tab se apni lambi UID copy karein aur 'admins' collection mein daalein (role = admin). Ab aap admin hain.

Chapter 8 — Kya Yeh Muft Hai: Ji haan. Spark (muft) plan mein rozana 50,000 reads, 20,000 writes, 1 GB storage — ek library app ke liye kaafi hai. Credit card ki zaroorat nahi.

Chapter 9-10 — AdMob (Ishtihaar se Kamai): admob.google.com kholein, wohi Gmail se sign in, 'Add app' → Android → naam daalein. Aapko **App ID** (jis mein ~ ka nishaan) milega. Phir 'Ad units' → 'Add ad unit' → **Banner** chunein → **Ad unit ID** (jis mein / ka nishaan) milega. Dono IDs developer ko dein. Code mein google_mobile_ads package, AndroidManifest mein App ID, aur ek 'AdaptiveBanner' widget lagana hai jo HAR screen size ke mutabiq khud fit ho jata hai (responsive). ZAROORI: test ke waqt hamesha TEST id istemal karein, apne ads par KHUD kabhi click na karein warna account band ho sakta hai.

Chapter 11 — Aakhri Checklist: Project banaya, app register ki, google-services.json diya, database banaya, Rules paste ki, login on kiya, apni UID admins mein daali, ek book add ki. AdMob: account banaya, App ID + Ad unit ID liya, code mein lagaya. Sab tick ho jaye to jo bhi ADMIN upload karega woh SAB users ko foran nazar aayega — yehi woh responsive system hai jo aap ne manga tha.

Note: Yeh app abhi mukammal offline chalti hai (phone ke andar). Jab aap upar diye gaye tamaam steps mukammal kar lein aur naya google-services.json app mein laga dein, tab online database jud jaega. Koi jaldi nahi — ek waqt mein ek step karein.

— Tayyar karda: A.A.A. COMMUNITY —